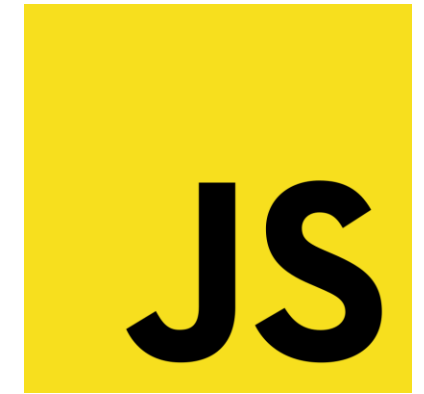


JavaScript 2025/1

FACOM32603 - Desenvolvimento Web I

Prof. Ian R. Cunha
ianresende@ufu.br



O que é JavaScript?

- É uma linguagem de programação
 - Não confundir com a linguagem de programação Java!
 - Comumente referenciada como JS
- Utilizada para prover **interatividade** e **dinamismo** a websites
- Comumente executada no lado cliente, pelo navegador Web
 - Linguagem interpretada pelo navegador
 - Não é necessário compilar explicitamente o código JavaScript
- Também pode ser utilizada no lado servidor
 - Utilizando ferramentas como o Node.js

O que é JavaScript?

- Permite **programar o comportamento** da página Web na ocorrência de eventos
 - Executar ações para inicializar a página Web assim que ela é carregada
 - Executar operações em resposta a ações do usuário, como “clicar em um botão” ou “selecionar uma opção”
 - Validar o conteúdo de campos de formulários à medida que o usuário os preenche
- Permite **alterar o documento HTML** por meio da manipulação da árvore DOM
 - Alterar o conteúdo e o layout da página Web em tempo de exibição
 - Adicionar novos elementos, ocultar/exibir elementos, modificar atributos de elementos, ...

Execução do código JavaScript no navegador

- Cada navegador tem seu próprio motor JavaScript
 - Google Chrome: V8
 - Microsoft Edge: V8 (antigamente usava o Chakra)
 - Firefox: SpiderMonkey
 - Safari: JavaScriptCore
- Quando o motor JavaScript escaneia o arquivo com o código (script), um ambiente chamado de **contexto de execução** é criado
 - Nesse momento, **memória é alocada** para as variáveis e funções
 - Dois tipos de contexto de execução: **global** e de **função**

Fases do contexto de execução JavaScript

Fases aplicáveis tanto no contexto global quanto de função.

1. Fase de **criação**

- O motor JS cria o contexto de execução e configura o ambiente do script. Determina os valores de variáveis e funções e configura a cadeia de escopo para o contexto de execução
- O motor JS associa o documento HTML no objeto `document` e a janela (do navegador) é associada ao objeto `window`

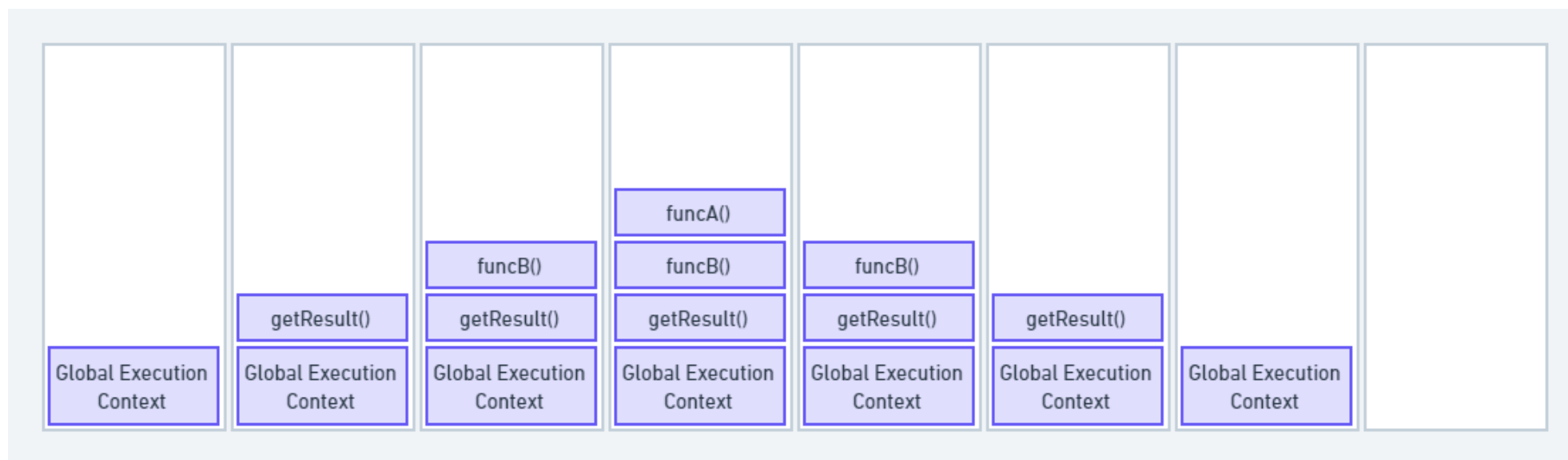
2. Fase de **execução**

- O motor JS executa o código no contexto de execução, linha por linha, de cima para baixo. Ele processa quaisquer instruções ou expressões no script e avalia quaisquer chamadas de função

Cada navegador tem seu tamanho fixo de pilha. Se o número de contexto excede o tamanho, então ocorre um erro de *stack overflow*.

Pilha de contexto de execução

- Para acompanhar todos os contextos, inclusive globais e funcionais, o motor JavaScript usa uma pilha de chamadas
- Usa o princípio LIFO (Last-In-First-Out), ou seja, a última execução a ser empilhada é a primeira a ser desempilhada



Formas de execução

1. Execução **síncrona**

- O motor JS executa sequencialmente e já retorna o resultado
- Uma instrução tem que esperar que a instrução anterior seja executada

2. Execução **assíncrona**

- Faz uma chamada para executar um trecho de código, mas não fica esperando o resultado (não bloqueia a execução do restante do código)
- Após toda a execução do código, o trecho é colocado na pilha de chamadas e, finalmente, é executado
- Código executado na ocorrência de eventos como click de botão, rolagem da página, dado de rede disponível etc.
- As funções a serem executadas geralmente são registradas na fase de criação do contexto

JavaScript no navegador - *Thread*

- O navegador executa o código JS em modo de ***thread* única** (*single-threaded*)
 - Durante a execução do código JS, o navegador não responde à interface do usuário (exceto em requisições Ajax)
 - Pode causar uma experiência ruim de navegação ao executar operações que demoram para finalizar
- É possível executar código JS em *background* (outra *thread*) usando *web workers*, mas com acesso limitado ao contexto da *thread* principal

Incorporar código JS na página HTML

Duas maneiras:

1. Embutido no HTML

- Mais simples e direto
- Código acessível somente dentro de uma página específica
- Dificulta a manutenção de sistemas maiores

2. Em arquivo separado

- Melhor separação de responsabilidades entre conteúdo (HTML) e comportamento (código JS)
- HTML conciso - código mais fácil de ler e manter
- Possibilita reutilizar o código JS em vários arquivos HTML
- Arquivos JavaScript podem ser mantidos em cache pelo navegador

Código JavaScript embutido no HTML

<head>

<title>Aqui vai o título da página</title>

<script type="text/javascript">

//meu código Javascript

...

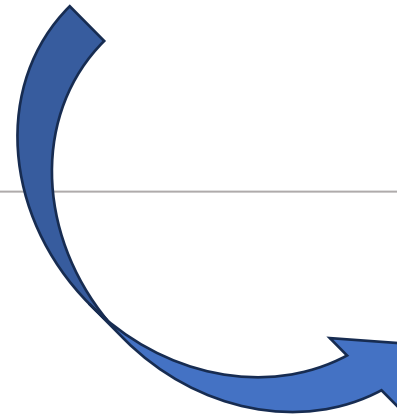
</script>

</head>

Código JavaScript em arquivo separado

pagina.html

```
...  
<head>  
  <title>Aqui vai o título da página</title>  
  <script src="meujavascript.js"></script>  
</head>  
...
```



meujavascript.js

```
//meu código Javascript  
  
...
```

Observações gerais sobre a linguagem

- JavaScript é sensível a maiúsculas e minúsculas (*case-sensitive*)
- Declarações podem ou não terminar com o ponto-e-vírgula
- Os tipos das variáveis são definidos automaticamente
 - linguagem dinâmica com tipos dinâmicos
 - linguagem de tipagem fraca
- Comentários de linha: `// comentário`
- Comentários de bloco: `/* comentário */`

Algumas propriedades e métodos gerais

window: Representa a aba do navegador que contém a página e possibilita obter informações ou realizar ações a respeito da janela

- `window.innerHeight` : retorna a altura do conteúdo da página
- `window.innerWidth` : retorna a largura do conteúdo da página
- `window.alert()` : mostra uma caixa de alerta com uma mensagem e um botão OK
- `window.confirm()` : mostra uma caixa de diálogo para confirmação (botões OK e cancelar)
- `window.close()` : fecha a janela

document (ou window.document) : Representa o documento HTML carregado na aba do navegador e possibilita a manipulação da árvore DOM

- `document.URL` : retorna a URL completa do documento HTML
- `document.title` : retorna o título da página HTML
- `document.write()` : escreve um texto no documento HTML

Algumas propriedades e métodos gerais

navigator (ou **window.navigator**): Representa o navegador de Internet em uso (browser, user-agent) e fornece informações como idioma do navegador, geolocalização, memória, etc.

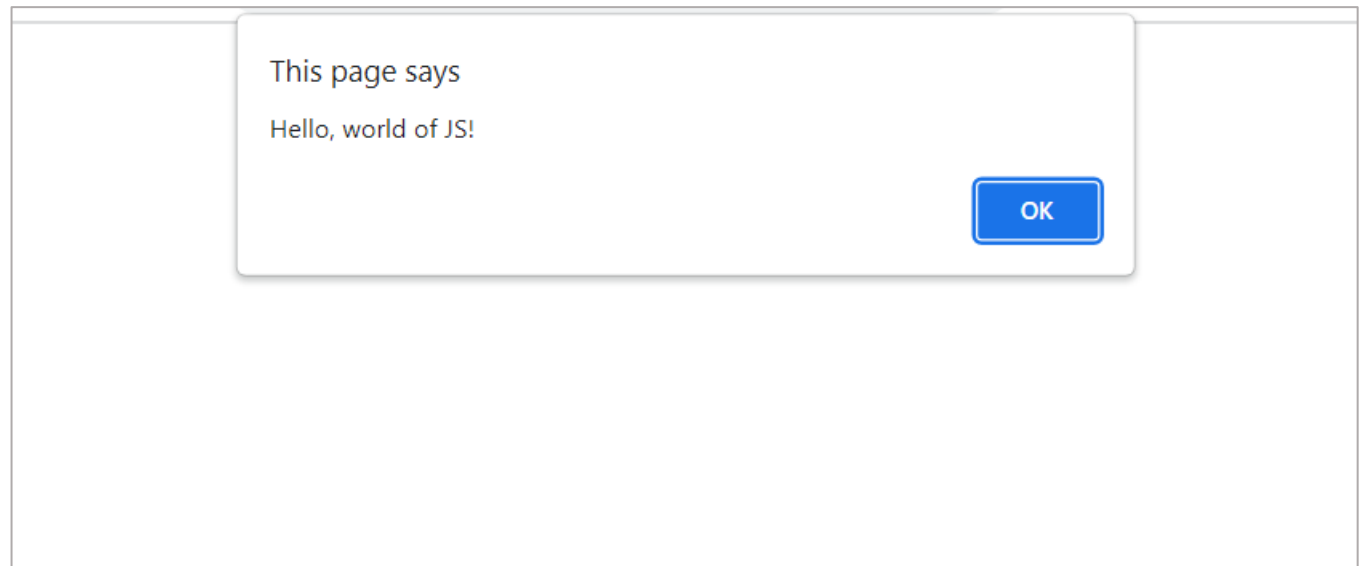
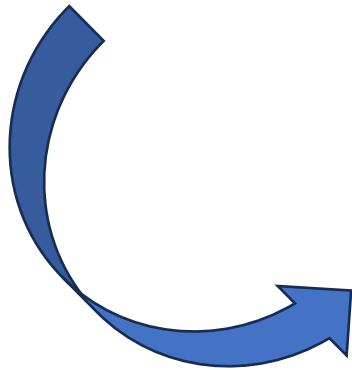
- `navigator.language`: retorna o idioma do navegador ("en", "en-US", "pt", "pt-BR", ...)
- `navigator.geolocation`: retorna um objeto de geolocalização para a localização do usuário
- `navigator.platform`: retorna a plataforma do navegador ("Linux i686", "Mac68K", "Win32", "WebTV OS", ...)

console (ou **window.console**): Representa o console de debug do navegador

- `console.log()`: registra conteúdo de log no console do navegador

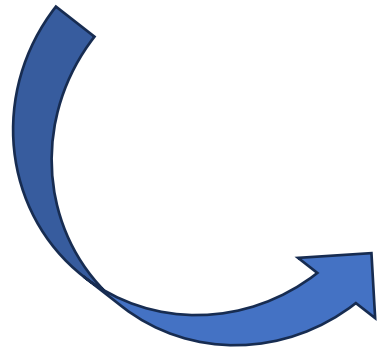
Exemplo 1

```
<script type="text/javascript">  
    alert("Hello, world of JS!");  
</script>
```



Exemplo 2

```
<script type="text/javascript">  
    document.write("A altura do conteúdo da janela é: " + window.innerHeight + "px. <br/>");  
    document.write("A largura do conteúdo da janela é: " + window.innerWidth + "px. <br/>");  
</script>
```



A altura do conteúdo da janela é: 739px.
A largura do conteúdo da janela é: 1536px.

Exemplo 3

```
<script type="text/javascript">  
    alert(document.URL);  
    alert(document.title);  
  
    document.write("A URL desta página é: " + document.URL);  
</script>
```

Variáveis

- Os tipos das variáveis são definidos automaticamente
 - linguagem dinâmica com tipos dinâmicos (podem ser alterados em tempo de execução)
 - linguagem de tipagem fraca, ou seja, não ocorre verificação de tipos nas operações efetuadas
- O nome deve começar por uma letra ou pelo caractere “_”
- Como já mencionado, é *case-sensitive* (teste, Teste e TESTE são diferentes)
- Podem ser definidas tanto no escopo global quanto de função (local)

Variáveis

- Não é necessário declarar o tipo de uma variável
- A variável irá “alterar” o seu tipo de dado conforme os valores forem atribuídos

```
var x; // x é indefinido  
x = 5; // x é um número  
x = "John"; // x é uma string  
x = true; // x é um valor lógico  
x = null; // x é indefinido
```

Declaração de variáveis

var nomeDaVariavel = valorInicial

- Variável com escopo local se declarada dentro de uma função
- Variável com escopo global se declarada fora de funções
- Pode ser redeclarada e pode ter valor atualizado
- Variáveis globais também podem ser acessadas pelo objeto window

let nomeDaVariável = valorInicial

- Variável tem escopo restrito ao bloco de código
- Pode ser acessada e atualizada apenas dentro do bloco
- Não pode ser redeclarada no mesmo bloco

Blocos de código
são delimitados
por chaves: { }

const nomeDaConstante = valor

- Semelhante a let, mas não pode ser atualizada
- Deve ser inicializada no momento da declaração

Operadores aritméticos e atribuição

Operação	Operador
Adição (e concatenação)	+
Subtração	-
Multiplicação	*
Divisão	/
Resto da divisão inteira	%
Exponenciação	**
Incremento	++
Decremento	--
Atribuição	=
Atribuição com soma	+=
Atribuição com subtração	-=

$x++$ é o mesmo que $x = x + 1$

$x--$ é o mesmo que $x = x - 1$

$x += 10$ é o mesmo que $x = x + 10$

$x -= 10$ é o mesmo que $x = x - 10$

Operadores de relacionais e lógicos

Operação	Operador
Comparação por igualdade	==
Diferente	!=
Estritamente igual (valor e tipo)	===
Estritamente diferente	!==
Menor que	<
Menor ou igual a	<=
Maior que	>
Maior ou igual a	>=

Operação	Operador
E	&&
OU	
NOT	!
Coalescência nula	??

Retorna o operando do lado direito quando o seu operador do lado esquerdo é null ou undefined. Caso contrário, retorna o operando do lado esquerdo.
Ex.: `const foo = null ?? 'default string';`

Operador de adição e concatenação

- O operador **+** deve ser utilizado com atenção
- Possibilita **somar** ou **concatenar**, dependendo dos operandos
- Se os dois operandos são numéricos então é realizada a soma
- Se um dos operandos é uma string então será feita a concatenação
 - O outro operando é convertido para string, caso não seja
- Exemplos:
 - `let x = 5 + 5; // x terá o valor 10`
 - `let y = "5" + 5; // y terá a string "55"`

Operadores == e ===

- Operador `==`
 - Compara apenas valores (conteúdo)
 - Operandos de tipos diferentes são convertidos e valores comparados
- Operador `===`
 - Compara o valor e o tipo dos operandos
 - Operandos de tipos diferentes sempre resulta em falso
- Exemplos:
 - `1 == true; // retorna true, pois true é convertido para 1`
 - `1 === true; // retorna false;`
 - `10 == "10" // retorna true depois de converter 10 para string`
 - `10 === "10" // retorna false;`

Controle condicional (if-else)

- Uma declaração condicional é um conjunto de comandos que são executados caso uma condição especificada seja verdadeira

```
if (condicao) {  
    //instrução 1  
    //instrução 2  
} else {  
    //instrução 3  
    //instrução 4  
}
```

```
if (condicao) {  
    //instrução 1  
    //instrução 2  
} else if (condição_2) {  
    //instrução 3  
    //instrução 4  
} else {  
    //instrução 5  
    //instrução 6  
}
```

Controle condicional (switch-case)

- Uma declaração switch permite que um programa avalie uma expressão e tente associar o valor da expressão ao rótulo de um case. Se uma correspondência for encontrada, o programa executa a declaração associada.

```
switch (expressao) {  
    case rotulo_1:  
        //instrução 1  
        //instrução 2  
        break;  
    case rotulo_2:  
        //instrução 3  
        //instrução 4  
        //break;  
    ...  
    default:  
        //instrução y  
        //instrução z  
        //break;  
}
```

Controle de repetição

- Laços (loops) oferecem um jeito fácil e rápido de executar uma ação repetidas vezes
- Várias formas de implementação

```
for (var i = 0; i < 5; i++) {  
    ....  
}
```

```
while (i < 5) {  
    i++;  
    ...  
}
```

```
do {  
    i += 1;  
    ...  
} while (i < 5);
```

Exemplo 4 - repetição

```
<script type="text/javascript">  
    for (var i = 0; i < 5; i++) {  
        alert("Hello, world " + i + "!");  
    }  
</script>
```

Arrays

- Não são tipos de dados primitivos
- São tratados como objetos, com propriedades e métodos
 - Por exemplo, o número de elementos pode ser resgatado por meio da propriedade `length`
- Podem armazenar valores de tipos diferentes
- Os elementos são acessados por índice numérico
 - O primeiro elemento do array possui índice 0

Arrays

- Elementos colocados entre colchetes, separados por vírgula
 - `let pares = [2, 4, 6, 8];`
 - `let primeiroPar = pares[0]; // 1º elemento`
 - `let nroElementos = pares.length; // tamanho do vetor`
- Elementos de diferentes tipos
 - `let vetorMisto = [2, 'A', true];`
- Pode ser iniciado com vazio
 - `let pares = [];`
 - `let pares = array();`

Exemplo 5 - arrays

```
var vetorDeNumeros = [4, 1, 5, 2, 6];  
var soma = 0;  
var n = vetorDeNumeros.length;  
  
for (var i = 0; i < n; i++)  
    soma = soma + vetorDeNumeros[i];  
  
alert("A soma dos elementos do vetor é: " + soma);
```

Arrays – alguns métodos

```
let vogais = ['E', 'I', 'O'];
```

- `vogais.push('U')` : adiciona um item no **final** do vetor
- `vogais.pop()` : remove e retorna o **último** item do vetor
- `vogais.shift()` : remove e retorna o **primeiro** item do vetor
- `vogais.unshift('A')` : adiciona um item no **início** do vetor
- `vogais.indexOf('E')` : retorna a posição da 1ª ocorrência de um item (ou -1)

Percorrendo arrays

- É possível percorrer utilizando as estruturas **for** e **forEach**

```
let pares = [2, 4, 6, 8];  
  
for (let i = 0; i < pares.length; i++) {  
    console.log(pares[i]);  
};
```

```
let pares = [2, 4, 6, 8];  
for (let item of pares) {  
    console.log(item);  
};
```

```
let pares = [2, 4, 6, 8];  
let soma = 0;  
pares.forEach( function (elemento) {  
    soma += elemento;  
});
```

Objeto Math

Método	Descrição
Math.sqrt(x)	Retorna a raiz quadrada de x.
Math.pow(x,y)	Retorna o valor de x elevado a y.
Math.PI	Retorna o valor da constante matemática PI.
Math.sin(x)	Calcula o seno do angulo x, dado em radianos.
Math.cos(x)	Calcula o cosseno do angulo x, dado em radianos.
Math.tan(x)	Calcula a tangente do angulo x, dado em radianos.
Math.round(x)	Retorna o valor de x arredondado para o inteiro mais próximo.
Math.random()	Retorna um número fracionário aleatório entre 0 e 1.

Objeto simples (plain object)

- Plain Old JavaScript Object (POJO)
- Contém apenas dados
- Pode ser definido utilizando chaves { }
- Possui lista de pares do tipo **propriedade : valor**
- Criado como instância da classe **Object**

```
let carro = {  
  modelo: "Fusca",  
  ano: 1970,  
  cor: "bege",  
  "motor-hp": 65  
}  
console.log(carro.ano); // 1970  
console.log(carro["motor-hp"]); // 65
```

Strings

- Definida com aspas simples ou duplas
 - `let msg = "JavaScript";`
- Acessando um caracter
 - `let primeiraLetra = msg[0];`
 - `let primeiraLetra = msg.charAt(0);`
- Contra-barra para caracteres especiais
 - `let msg = 'It\'s ok';`
- Strings com aspas duplas podem conter aspas simples e vice-versa
 - `let msg = "It's ok";`
- Várias propriedades e métodos
 - `length`, `indexOf`, `substr`, `split`, etc.

Template Literal (ou Template String)

- Strings definidas com o caractere crase (*backtick*)
 - ``minha string``
- Suporta fácil interpolação de variáveis e expressões usando `${ }`
- Maior facilidade para definir strings de múltiplas linhas
- A string pode conter aspas simples ou duplas

```
let a = 1;
let b = 2;
let c = 3;
const delta = b*b - 4*a*c;
console.log(`o discriminante da equacao com coeficientes ${a}, ${b} e ${c} é ${delta}`);
```

Funções

- São trechos de códigos comuns criados para serem reutilizados
- Exige a palavra reservada `function`
- Exemplos:
 - função para somar dois números
 - função para calcular o valor do frete de um produto
 - função para ordenar uma lista
 - função para salvar um dado em um banco
 - função para verificar a disponibilidade de um recurso

Declaração de funções

```
function nomeDaFuncao(par1, par2, par3, ...) {  
    // operações  
    // operações  
    // operações  
}
```

```
function max(a, b) {  
    if (a > b)  
        return a;  
    else  
        return b;  
}  
  
let maior = max(2, 5);
```

Quando 'return' não é utilizada, o valor undefined é automaticamente retornado.

Funções anônimas

- É simplesmente uma função sem nome
- Forma muito utilizada para tratar eventos e para passar uma função como parâmetro de outra

Função tradicional

```
function calculaQuadrado(numero) {  
    return numero * numero;  
}
```

A variável dá um nome para uma função.

Função anônima

```
var quadrado = function (numero) {  
    return numero * numero;  
};
```


Exercício 1

- Implemente uma função em JS para somar dois números.
- Utilize a função `window.prompt()` para receber cada um dos números do usuário.
 - Será necessário utilizar a função `parseInt()` para converter o número do formato de string para inteiro
- Utilize a função `window.alert()` para exibir o resultado.

Exercício 2

- Altere o exercício para receber vários números a serem somados.
- O usuário deverá digitar a letra 's' para parar de informar valores.
- Utilize um array para armazenar os valores.

Referências

- <https://developer.mozilla.org/en-US/docs/Web/JavaScript>
- <https://www.ecma-international.org/ecma-262/>
- Agradecimento especial ao Prof. Daniel Furtado pela disponibilização do material: <https://furtado.prof.ufu.br>